

[FastAPI](#) [Learn](#) [Tutorial - User Guide](#)

SQL (Relational) Databases

FastAPI doesn't require you to use a SQL (relational) database. But you can use **any database** that you want.

Here we'll see an example using [SQLModel](#).

SQLModel is built on top of [SQLAlchemy](#) and Pydantic. It was made by the same author of **FastAPI** to be the perfect match for FastAPI applications that need to use **SQL databases**.

Tip

You could use any other SQL or NoSQL database library you want (in some cases called "ORMs"), FastAPI doesn't force you to use anything. 😊

As SQLModel is based on SQLAlchemy, you can easily use **any database supported** by SQLAlchemy (which makes them also supported by SQLModel), like:

- PostgreSQL
- MySQL
- SQLite
- Oracle
- Microsoft SQL Server, etc.

In this example, we'll use **SQLite**, because it uses a single file and Python has integrated support. So, you can copy this example and run it as is.

Later, for your production application, you might want to use a database server like **PostgreSQL**.

Tip

There is an official project generator with **FastAPI** and **PostgreSQL** including a frontend and more tools:
<https://github.com/fastapi/full-stack-fastapi-template>

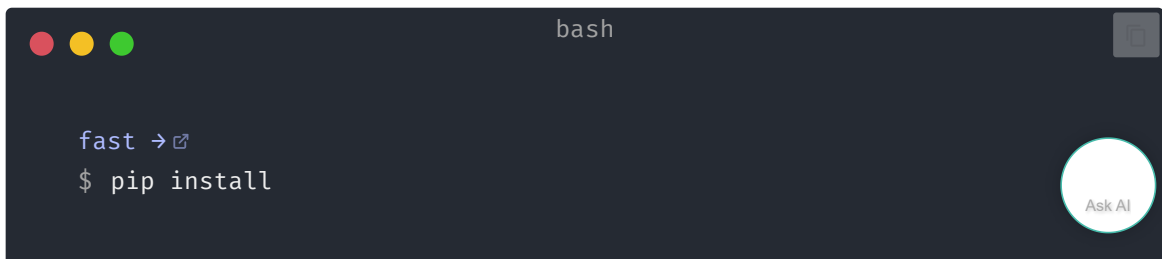
This is a very simple and short tutorial, if you want to learn about databases in general, about SQL, or more advanced features, go to the [SQLModel docs](#).

Install `SQLModel`

First, make sure you create your [virtual environment](#), activate it, and then install `sqlmodel` :

```
bash

fast → 
$ pip install
```



Create the App with a Single Model

We'll create the simplest first version of the app with a single **SQLModel** model first.

Later we'll improve it increasing security and versatility with **multiple models** below. 🤖

Create Models

Import `SQLModel` and create a database model:

Python 3.10.0 +

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str
```

Code below omitted 📌

Full file preview

Python 3.10.0 +

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

The `Hero` class is very similar to a Pydantic model (in fact, underneath, it actually *is* a *Pydantic model*).

There are a few differences:

- `table=True` tells `SQLModel` that this is a *table model*, it should represent a **table** in the SQL database, it's not just a *data model* (as would be any other regular Pydantic class).
- `Field(primary_key=True)` tells `SQLModel` that the `id` is the **primary key** in the SQL database (you can learn more about SQL primary keys in the `SQLModel` docs).

Note: We use `int | None` for the primary key field so that in Python code we can *create an object without an `id`* (`id=None`), assuming the database will *generate it when saving*. `SQLModel` understands that the database will provide the `id` and *defines the column as a non-null `INTEGER`* in the database schema. See [SQLModel docs on primary keys](#) for details.

- `Field(index=True)` tells `SQLModel` that it should create a **SQL index** for this column, that would allow faster lookups in the database when reading data filtered by this column.

`SQLModel` will know that something declared as `str` will be a SQL column of type `TEXT` (or `VARCHAR`, depending on the database).

Create an Engine

A `SQLModel engine` (underneath it's actually a `SQLAlchemy engine`) is what **holds the connections** to the database.

You would have **one single engine object** for all your code to connect to the same database.

Python 3.10+

```
# Code above omitted 🙌

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

# Code below omitted 🙌
```

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"


connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Using `check_same_thread=False` allows FastAPI to use the same SQLite database in different threads. This is necessary as **one single request** could use **more than one thread** (for example in dependencies).

Don't worry, with the way the code is structured, we'll make sure we use **a single SQLAlchemy session per request** later, this is actually what the `check_same_thread` is trying to achieve.

Create the Tables

We then add a function that uses `SQLModel.metadata.create_all(engine)` to **create the tables** for all the *table models*.

Python 3.10 +

Code above omitted 👉

```
def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
```

Code below omitted 👉

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Create a Session Dependency

A **Session** is what stores the **objects in memory** and keeps track of any changes needed in the data, then it **uses the engine** to communicate with the database.

We will create a FastAPI **dependency** with `yield` that will provide a new **Session** for each request. This is what ensures that we use a single session per request. 🤖

Then we create an `Annotated` dependency `SessionDep` to simplify the rest of the code that will use this dependency.

Python 3.10+

Code above omitted 🖱️

```
def get_session():
    with Session(engine) as session:
        yield session
```

```
SessionDep = Annotated[Session, Depends(get_session)]
```

Code below omitted 🖱️

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:////{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")
```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Create Database Tables on Startup

We will create the database tables when the application starts.

Python 3.10+

Code above omitted 🖱

app = FastAPI()

@app.on_event("startup")

def on_startup():

create_db_and_tables()

Code below omitted 🖱

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:////{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Here we create the tables on an application startup event.

For production you would probably use a migration script that runs before you start your app. 🤖

Tip

SQLModel will have migration utilities wrapping Alembic, but for now, you can use [Alembic](#) directly.

Create a Hero

Because each SQLModel model is also a Pydantic model, you can use it in the same **type annotations** that you could use Pydantic models.

For example, if you declare a parameter of type `Hero`, it will be read from the **JSON body**.

The same way, you can declare it as the function's **return type**, and then the shape of the data will show up in the automatic API docs UI.

Python 3.10 +

Code above omitted 🤖

```
@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero
```

Code below omitted 🤖

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Here we use the `SessionDep` dependency (a `Session`) to add the new `Hero` to the `Session` instance, commit the changes to the database, refresh the data in the `hero`, and then return it.

Read Heroes

We can **read** `Hero`s from the database using a `select()`. We can include a `limit` and `offset` to paginate the results.

Python 3.10+

```
# Code above omitted 🖱️

@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

# Code below omitted 🖱️
```

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Read One Hero

We can **read** a single Hero.

Python 3.10 +

```
# Code above omitted 👉

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

# Code below omitted 👉
```

Full file preview

Python 3.10+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.delete("/heroes/{hero_id}")

```

```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Delete a Hero

We can also **delete** a `Hero`.

Python 3.10 +

```
# Code above omitted 🙌

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Full file preview

Python 3.10.0+

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero


@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(100)] = 100,
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
```

```
session.delete(hero)
session.commit()
return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(get_session)) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
) -> list[Hero]:
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: Session = Depends(get_session)) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

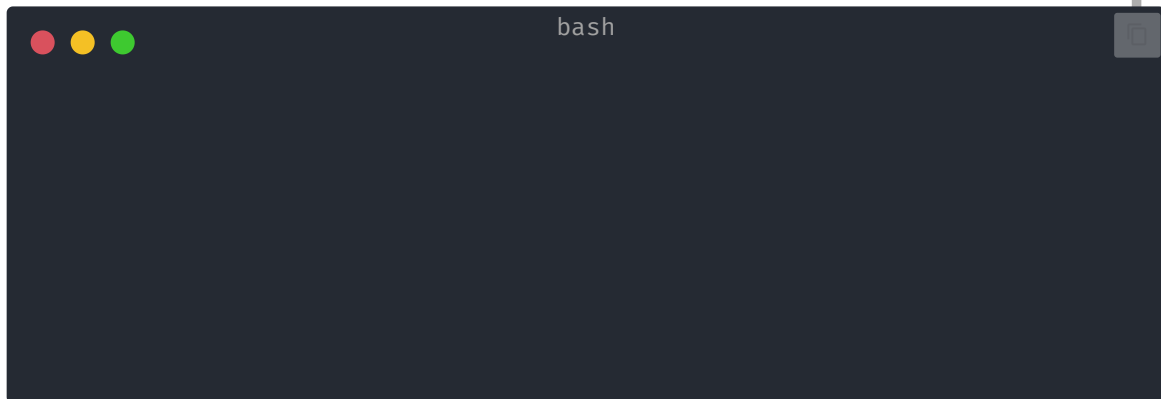
@app.delete("/heroes/{hero_id}")

```

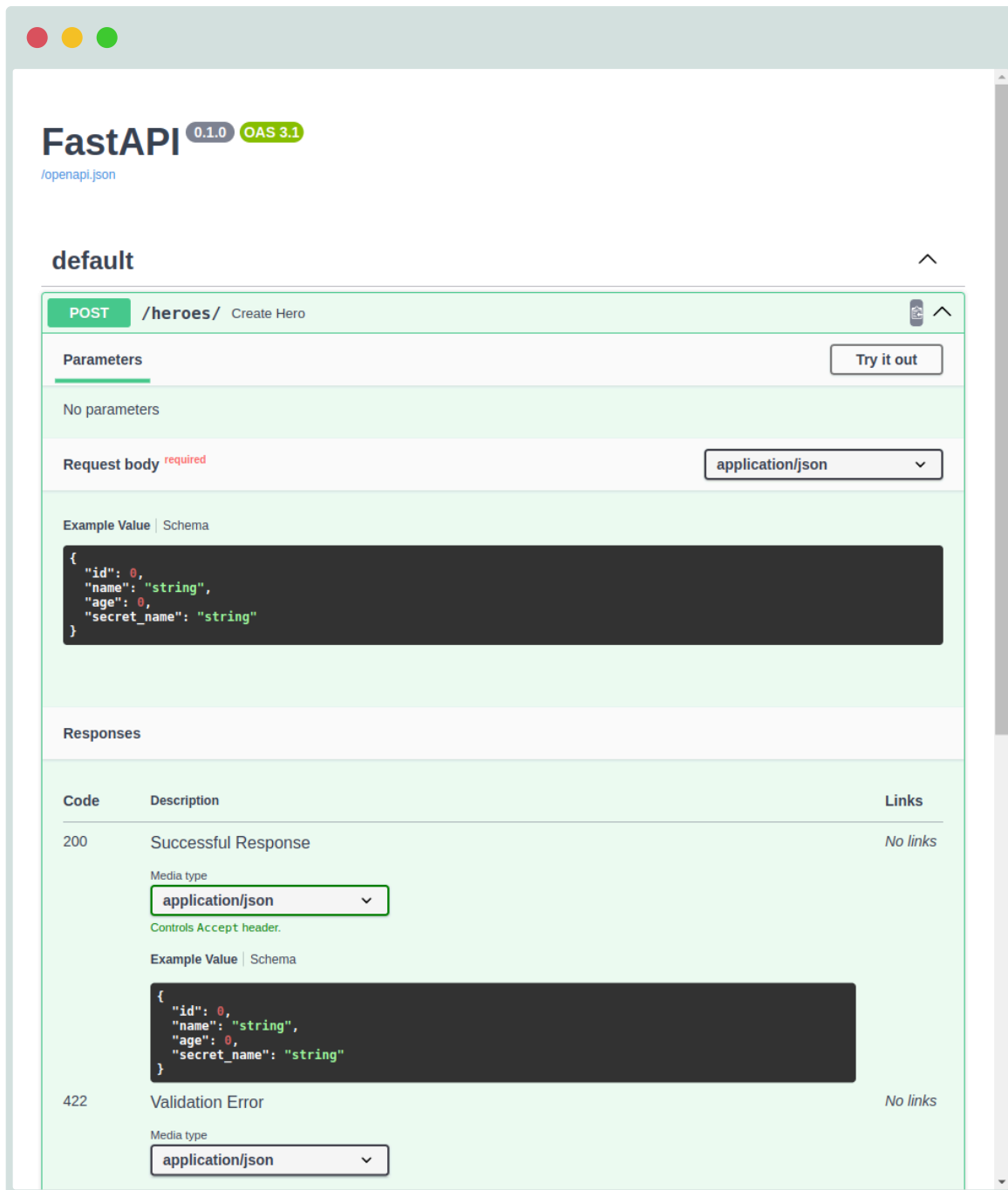
```
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Run the App

You can run the app:



Then go to the `/docs` UI, you will see that **FastAPI** is using these **models** to **document** the API, and it will use them to **serialize** and **validate** the data too.



Update the App with Multiple Models

Now let's **refactor** this app a bit to increase **security** and **versatility**.

If you check the previous app, in the UI you can see that, up to now, it lets the client decide the `id` of the `Hero` to create. 🙄

We shouldn't let that happen, they could overwrite an `id` we already have assigned in the DB. Deciding the `id` should be done by the **backend** or the **database**, **not by the client**.

Additionally, we create a `secret_name` for the hero, but so far, we are returning it everywhere, that's not very **secret**... 😏

We'll fix these things by adding a few **extra models**. Here's where SQLAlchemy will shine. ✨

Create Multiple Models

In **SQLModel**, any model class that has `table=True` is a **table model**.

And any model class that doesn't have `table=True` is a **data model**, these ones are actually just Pydantic models (with a couple of small extra features). 🤖

With SQLModel, we can use **inheritance** to **avoid duplicating** all the fields in all the cases.

HeroBase - the base class

Let's start with a `HeroBase` model that has all the **fields that are shared** by all the models:

- `name`
- `age`

Python 3.10 +

```
# Code above omitted 👉  
  
class HeroBase(SQLModel):  
    name: str = Field(index=True)  
    age: int | None = Field(default=None, index=True)  
  
# Code below omitted 👉
```

Full file preview

Python 3.10+

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Hero - the *table model*

Then let's create `Hero`, the actual *table model*, with the **extra fields** that are not always in the other models:

- `id`
- `secret_name`

Because `Hero` inherits from `HeroBase`, it **also** has the **fields** declared in `HeroBase`, so all the fields for `Hero` are:

- `id`
- `name`
- `age`
- `secret_name`

Python 3.10+

```

# Code above omitted 🙌

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

```

```
# Code below omitted 📌
```

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

HeroPublic - the public data model

Next, we create a `HeroPublic` model, this is the one that will be **returned** to the clients of the API.

It has the same fields as `HeroBase`, so it won't include `secret_name`.

Finally, the identity of our heroes is protected! 🙋

It also re-declares `id: int`. By doing this, we are making a **contract** with the API clients, so that they can always expect the `id` to be there and to be an `int` (it will never be `None`).

Tip

Having the return model ensure that a value is always available and always `int` (not `None`) is very useful for the API clients, they can write much simpler code having this certainty.

Also, **automatically generated clients** will have simpler interfaces, so that the developers communicating with your API can have a much better time working with your API. 😎

All the fields in `HeroPublic` are the same as in `HeroBase`, with `id` declared as `int` (not `None`):

- `id`
- `name`
- `age`

Python 3.10.0 +

Code above omitted 🖱️

```
class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str
```

```
class HeroPublic(HeroBase):
    id: int
```

Code below omitted 🖱️

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

HeroCreate - the *data model* to create a hero

Now we create a `HeroCreate` model, this is the one that will **validate** the data from the clients.

It has the same fields as `HeroBase`, and it also has `secret_name`.

Now, when the clients **create a new hero**, they will send the `secret_name`, it will be stored in the database, but those secret names won't be returned in the API to the clients.

Tip

This is how you would handle **passwords**. Receive them, but don't return them in the API.

You would also **hash** the values of the passwords before storing them, **never store them in plain text**.

The fields of `HeroCreate` are:

- `name`
- `age`
- `secret_name`

Python 3.10 +

Code above omitted 🙌

```
class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str
```

Code below omitted 📌

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

HeroUpdate - the *data model* to update a hero

We didn't have a way to **update a hero** in the previous version of the app, but now with **multiple models**, we can do it.



The `HeroUpdate` *data model* is somewhat special, it has **all the same fields** that would be needed to create a new hero, but all the fields are **optional** (they all have a default value). This way, when you update a hero, you can send just the fields that you want to update.

Because all the **fields actually change** (the type now includes `None` and they now have a default value of `None`), we need to **re-declare** them.

We don't really need to inherit from `HeroBase` because we are re-declaring all the fields. I'll leave it inheriting just for consistency, but this is not necessary. It's more a matter of personal taste. 🙄

The fields of `HeroUpdate` are:

- `name`
- `age`
- `secret_name`

Python 3.10+

Code above omitted 🙄

```
class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None
```

Code below omitted 📌

Full file preview

Python 3.10+

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Create with `HeroCreate` and return a `HeroPublic`

Now that we have **multiple models**, we can update the parts of the app that use them.

We receive in the request a `HeroCreate` *data model*, and from it, we create a `Hero` *table model*.

This new *table model* `Hero` will have the fields sent by the client, and will also have an `id` generated by the database.

Then we return the same *table model* `Hero` as is from the function. But as we declare the `response_model` with the `HeroPublic` *data model*, **FastAPI** will use `HeroPublic` to validate and serialize the data.

Python 3.10 +

```

# Code above omitted 📄

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

# Code below omitted 📄

```

Full file preview

Python 3.10+

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```



Tip

Now we use `response_model=HeroPublic` instead of the **return type annotation** `-> HeroPublic` because the value that we are returning is actually *not* a `HeroPublic`.

If we had declared `-> HeroPublic`, your editor and linter would complain (rightfully so) that you are returning a `Hero` instead of a `HeroPublic`.

By declaring it in `response_model` we are telling **FastAPI** to do its thing, without interfering with the type annotations and the help from your editor and other tools.

Read Heroes with `HeroPublic`

We can do the same as before to **read** Heroes, again, we use `response_model=list[HeroPublic]` to ensure that the data is validated and serialized correctly.

Python 3.10+

```

# Code above omitted 🐡

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,

```

```
    ):  
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()  
        return heroes  
  
# Code below omitted 📌
```

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Read One Hero with HeroPublic

We can **read** a single hero:

Python 3.10+

```

# Code above omitted 📄

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

# Code below omitted 📄

```

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Update a Hero with HeroUpdate

We can **update a hero**. For this we use an HTTP `PATCH` operation.

And in the code, we get a `dict` with all the data sent by the client, **only the data sent by the client**, excluding any values that would be there just for being the default values. To do it we use `exclude_unset=True`. This is the main trick.



Then we use `hero_db.sqlmodel_update(hero_data)` to update the `hero_db` with the data from `hero_data`.

Python 3.10 +

Code above omitted 🖱

```

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

```

Code below omitted 🖱

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```

def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Delete a Hero Again

Deleting a hero stays pretty much the same.

We won't satisfy the desire to refactor everything in this one. 😊

Python 3.10+

```

# Code above omitted 🙌

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}

```

Full file preview

Python 3.10.0 +

```

from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):

```

```
heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
return heroes

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

 Other versions and variants

Python 3.10+ - non-Annotated

 TipPrefer to use the `Annotated` version if possible.

```

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{"sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

@app.get("/heroes/", response_model=list[HeroPublic])

```

```
def read_heroes(
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes

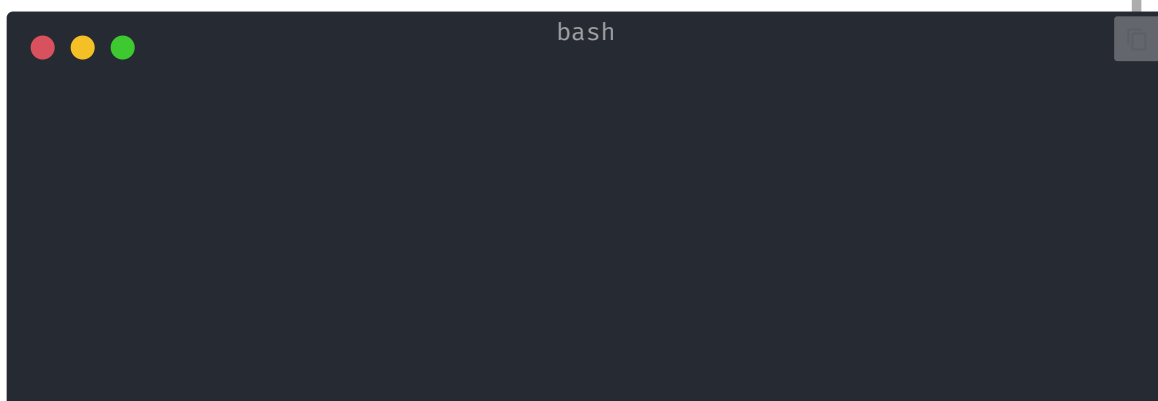
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    hero_id: int, hero: HeroUpdate, session: Session = Depends(get_session)
):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Run the App Again

You can run the app again:



If you go to the `/docs` API UI, you will see that it is now updated, and it won't expect to receive the `id` from the client when creating a hero, etc.

The screenshot shows the FastAPI Swagger UI for the `POST /heroes/` endpoint, titled "Create Hero". The interface includes a "Parameters" section with "No parameters", a "Request body" section with a required field and a dropdown set to `application/json`, and a "Responses" section. The "Request body" section shows an example value: `{ "name": "string", "age": 0, "secret_name": "string" }`. The "Responses" section lists two responses: a 200 "Successful Response" and a 422 "Validation Error", both with a media type dropdown set to `application/json`. The 200 response shows an example value: `{ "name": "string", "age": 0, "id": 0 }`.

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

POST /heroes/ Create Hero

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "name": "string",
  "age": 0,
  "secret_name": "string"
}
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json	
	Controls Accept header.	
	Example Value Schema	
	<pre>{ "name": "string", "age": 0, "id": 0 }</pre>	
422	Validation Error	No links
	Media type application/json	
	Example Value Schema	

Recap

You can use [SQLModel](#) to interact with a SQL database and simplify the code with *data models* and *table models*.

You can learn a lot more at the [SQLModel docs](#), there's a longer mini [tutorial on using SQLModel with FastAPI](#).